

JAVA DSA QUICK REFERENCE

Brief syntax and coding patterns for interview practice

Use this for Java syntax lookup while solving problems independently. The goal is to remove language friction - not to replace algorithmic reasoning.

Inside

- Arrays, strings and overflow-safe arithmetic
- Collections, maps, sets, queues, stacks and heaps
- Sorting, comparators and binary search
- Reusable tree, graph and dynamic-programming patterns
- Common Java mistakes and a coding-round checklist

For sustainable SDE-2 interview preparation

1. Java foundations

Use one import during DSA practice

```
import java.util.*;
```

Primitive types and safe arithmetic

```
int x = 10; // 32-bit
long y = 10L; // 64-bit
long sum = (long) a + b; // cast before overflow
long product = (long) a * b;
char ch = 'a';
boolean valid = true;

int max = Integer.MAX_VALUE;
long large = Long.MAX_VALUE;
```

Arrays

```
int[] nums = new int[n];
int[] values = {4, 1, 7};
int n = values.length;

Arrays.sort(values);
Arrays.fill(nums, -1);
int[] copy = Arrays.copyOf(values, values.length);
System.out.println(Arrays.toString(values));

for (int i = 0; i < values.length; i++) { }
for (int value : values) { }
```

2D arrays

```
int[][] grid = new int[rows][cols];
int rows = grid.length;
int cols = grid[0].length;

for (int[] row : grid) {
    Arrays.fill(row, -1);
}
```

Strings and StringBuilder

```
String s = "hello";
int n = s.length();
char first = s.charAt(0);
String part = s.substring(1, 4); // indices 1, 2, 3
boolean same = s.equals("hello"); // not ==
char[] chars = s.toCharArray();

StringBuilder sb = new StringBuilder();
sb.append('a').append(10);
sb.setCharAt(0, 'b');
sb.reverse();
String result = sb.toString();
```

2. Collections you will use repeatedly

Need	Java choice
Dynamic array	<code>ArrayList<Integer></code>
Value -> value/index	<code>HashMap<K, V></code>
Unique values	<code>HashSet<Integer></code>
Queue / stack	<code>ArrayDeque<Integer></code>
Min/max heap	<code>PriorityQueue<Integer></code>
Sorted map/set	<code>TreeMap</code> / <code>TreeSet</code>

ArrayList

```
List<Integer> list = new ArrayList<>();
list.add(10);
int value = list.get(0);
list.set(0, 20);
int size = list.size();
list.remove(list.size() - 1);    // remove index
list.remove(Integer.valueOf(10)); // remove value
```

HashMap

```
Map<Integer, Integer> frequency = new HashMap<>();

for (int value : nums) {
    frequency.put(value,
        frequency.getDefault(value, 0) + 1);
}

boolean exists = frequency.containsKey(key);
int count = frequency.getDefault(key, 0);

for (Map.Entry<Integer, Integer> entry : frequency.entrySet()) {
    int key = entry.getKey();
    int value = entry.getValue();
}
```

HashSet

```
Set<Integer> seen = new HashSet<>();
seen.add(value);
boolean exists = seen.contains(value);
seen.remove(value);

if (!seen.add(value)) {
    // value was already present
}
```

Queue and stack

```
Queue<Integer> queue = new ArrayDeque<>();
queue.offer(10); int front = queue.peek(); int x = queue.poll();

Deque<Integer> stack = new ArrayDeque<>();
stack.push(10); int top = stack.peek(); int y = stack.pop();
```

Prefer **offer** / **poll** / **peek** for queues. Prefer **ArrayDeque** over the older `Stack` class.

3. Sorting, heaps and binary search

Sorting

```
Arrays.sort(nums); // int[] ascending

// Sort rows by column 0
Arrays.sort(pairs,
    (a, b) -> Integer.compare(a[0], b[0]));

// Sort by first column, then second
Arrays.sort(pairs, (a, b) -> {
    if (a[0] != b[0]) return Integer.compare(a[0], b[0]);
    return Integer.compare(a[1], b[1]);
});

list.sort(Collections.reverseOrder());
```

Comparator rule

Return negative when **a should come before b**, zero when equal for ordering, and positive when **a should come after b**. Use `Integer.compare` instead of `a - b` to avoid overflow.

PriorityQueue

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>(Collections.reverseOrder());

PriorityQueue<int[]> byDistance = new PriorityQueue<>(
    (a, b) -> Integer.compare(a[0], b[0]));

byDistance.offer(new int[]{distance, node});
int[] current = byDistance.poll();
```

Binary search

```
int left = 0, right = nums.length - 1;

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] == target) return mid;
    if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
}

return -1;
```

Lower bound: first index with value $\geq$ target

```
int left = 0, right = nums.length;
while (left < right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] >= target) right = mid;
    else left = mid + 1;
}
return left; // may equal nums.length
```

4. Trees and graphs

Tree DFS

```
private void dfs(TreeNode node) {
    if (node == null) return;

    // preorder work
    dfs(node.left);
    // inorder work
    dfs(node.right);
    // postorder work
}
```

Level-order BFS

```
Queue<TreeNode> queue = new ArrayDeque<>();
if (root != null) queue.offer(root);

while (!queue.isEmpty()) {
    int levelSize = queue.size();

    for (int i = 0; i < levelSize; i++) {
        TreeNode node = queue.poll();
        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
}
```

Graph adjacency list

```
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

for (int[] edge : edges) {
    int u = edge[0], v = edge[1];
    graph.get(u).add(v);
    graph.get(v).add(u); // omit for directed graph
}
```

Graph BFS

```
boolean[] visited = new boolean[n];
Queue<Integer> queue = new ArrayDeque<>();
queue.offer(start);
visited[start] = true;

while (!queue.isEmpty()) {
    int node = queue.poll();
    for (int next : graph.get(node)) {
        if (!visited[next]) {
            visited[next] = true; // mark when enqueued
            queue.offer(next);
        }
    }
}
```

BFS/DFS complexity is **$O(V + E)$** with an adjacency list. Mark BFS nodes visited when enqueueing, not when removing them.

5. Dynamic programming and interview habits

Top-down memoization

```
private int solve(int index, int[] nums, Integer[] memo) {
    if (index >= nums.length) return 0;
    if (memo[index] != null) return memo[index];

    int skip = solve(index + 1, nums, memo);
    int take = nums[index] + solve(index + 2, nums, memo);

    return memo[index] = Math.max(skip, take);
}
```

Bottom-up and 2D initialization

```
int[] dp = new int[n + 1];
for (int i = 2; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2];
}

int[][] memo = new int[rows][cols];
for (int[] row : memo) Arrays.fill(row, -1);
```

DP thinking order

- Define exactly what each state represents.
- Write the recurrence before implementation.
- Identify base cases and invalid states.
- Time = number of states x work per state.
- Include the recursion stack in space complexity.

Common Java mistakes

Type	Size
Array	arr.length
String	s.length()
Collection	list.size()

- Compare String content with equals(), not ==.
- Collections use wrappers such as Integer, not primitive int.
- map.get(key) may return null; getOrDefault is often safer.
- Cast to long before an overflowing operation, not after it.
- ArrayDeque does not permit null elements.
- Deep recursive graph traversal may overflow Java's call stack.

Before submitting

- Explain the approach and invariant.
- State time and space complexity.
- Test empty/single input, duplicates, negatives and overflow.
- Check loop boundaries and null handling.
- Dry-run one normal case and one edge case.

Practice rule

Looking up Java syntax does not reduce diagnostic independence. Looking up the data structure, technique or solution does. Write in a plain editor first, compile second, and keep a short list of recurring syntax errors.